COMPUTER NETWORKS PROJECT

UPI Offline Mesh

Mesh Routed Deferred Payment Settlement

Project Guide: how to install, run, use, understand, test and demonstrate the system

Version 1.0.0 · 9 October 2026

> **Scope notice.** This is a software-simulated prototype. The mesh, the phones and the radio are simulated; all accounts and money are fictional; no bank, NPCI, real UPI network or Bluetooth hardware is involved. The cryptography, idempotency and settlement logic, however, are real implementations that are exercised by automated tests.

Contents

1. Project overview
2. Architecture
3. Technology stack
4. Installation and prerequisites
5. Complete execution guide
6. How to use the dashboard
7. Payment lifecycle
8. Networking concepts
9. Cryptography and security
10. Database and backend internals
11. API reference
12. Testing and verification
13. Troubleshooting
14. Viva preparation
15. Known limitations and future work

Figures are real screenshots of the running application. Command lines are written one per block so they can be pasted without comments.

1. Project overview

1.1 The problem

A normal digital payment assumes that the payer's phone can reach the bank's server at the moment of payment. That assumption fails in a basement, a crowded festival, a flooded street or a village with no signal. In those places the payer cannot connect, but other phones nearby might later walk into coverage. This project studies how a payment instruction can still travel from the payer to the backend by hopping through other devices, and how the backend can settle it safely when it finally arrives.

This is a **Computer Networks** problem before it is a payments problem. Three hard sub-problems appear as soon as relays are involved:

- **Trust.** A relay phone can read, copy, delay, drop or alter whatever it carries. The system must not depend on relays being honest.
- **Duplication.** Gossip forwarding deliberately makes many copies of a packet. Several bridge phones may upload the same packet to the backend at the same instant. Money must still move only once.
- **Replay.** An old, valid packet can be captured and sent again later. Encryption alone does not stop that.

1.2 What "mesh routed deferred payment settlement" means

- **Mesh routed:** the packet is passed between neighbouring devices (phone to phone) instead of travelling over one fixed path to a server.
- **Deferred:** settlement does not happen at payment time. The payment is stored and forwarded; it is settled later, when any bridge device gets internet.
- **Settlement:** the backend debits the sender, credits the receiver and writes a ledger entry in one database transaction.

1.3 Objectives

1. Demonstrate store-and-forward gossip with TTL, replication, packet loss, delay and intermittent gateways.
2. Keep payment contents confidential and tamper-evident while crossing untrusted relays.
3. Guarantee one ledger effect per logical payment however many copies arrive (idempotent settlement).
4. Measure how connectivity delay, packet loss, TTL, replication, bridge count and duplicate delivery change the outcome, with reproducible experiments.
5. Provide a dashboard and a test suite that let an evaluator see and verify every claim.

1.4 Scope and limitations of the simulation

The project is a **software simulation**. It is important to be precise about what is real and what is not:

Aspect	Status
AES-256-GCM, RSA-OAEP, SHA-256, nonce and timestamp checks	**Real.** JDK cryptography, no custom primitives.
Atomic settlement, unique constraints, row locks, concurrency control	**Real.** Spring Data JPA, Hibernate and H2.
Mesh devices, topology, gossip rounds, loss, delay, expiry	**Simulated** in Java (class `MeshNetwork`). Deterministic and seeded.
Phone-to-phone radio (Bluetooth, Wi-Fi Direct)	**Not implemented.** Packets move between in-memory objects.
Bridge to backend upload	**Simulated boundary.** The simulator calls the same Java service (`BridgeIngestionService`) that backs `POST /api/bridge/ingest`. That REST endpoint also exists and is tested over HTTP.
Sender phone	**Simulated** by `DemoService`. The encrypt-then-inject boundary is explicit.
UPI, NPCI, banks, real money, PIN verification	**Not involved at all.** All accounts and amounts are fictional.

> Why this is not a real UPI system: real UPI requires bank-grade identity, authentication, regulated settlement and an online balance check at payment time. This prototype has none of those. It demonstrates the networking and consistency ideas that an offline extension would need, nothing more.

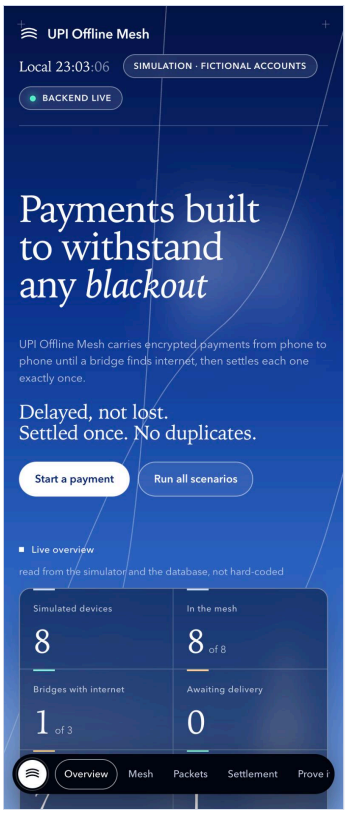


Figure 1.1: The dashboard landing view on a phone-width screen (live statistics are read from the backend).

2. Architecture

2.1 Runtime architecture

The application is one Spring Boot process. The dashboard is **not** a separate frontend project: `index.html`, `app.css` and `app.js` are static files in `src/main/resources/static` and are served by the same embedded Tomcat that serves the REST API. There is therefore a single command to start everything and no Node.js build step.

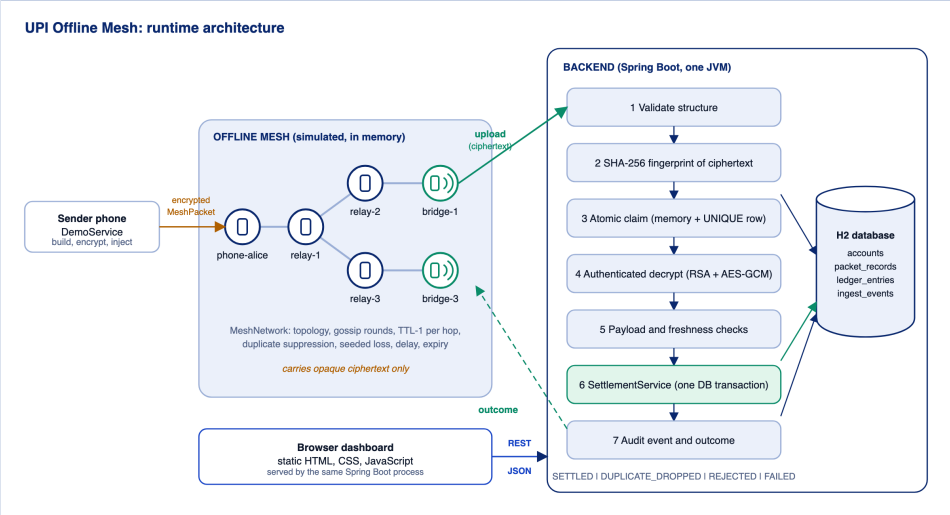


Figure 2.1: Runtime architecture: offline mesh, backend ingestion pipeline, database and dashboard.

2.2 Components and responsibilities

Component	Class(es)	Responsibility
Sender	`DemoService`, `HybridCryptoService.encrypt`	Builds the payment instruction (sender, receiver, amount, nonce, signedAt), encrypts it with the server public key, wraps it in a `MeshPacket` and injects it into the sender device. Also simulates attackers (tamper, stale, replay).
Virtual devices	`VirtualDevice`, `StoredPacket`	A simulated phone: role (RELAY or BRIDGE), in-mesh flag, internet flag, and a store of packet copies with TTL, hop count and upload state.
Mesh simulator	`MeshNetwork`, `Topology`, `MeshSettings`	Pure Java network model: neighbour relationships, gossip rounds, TTL decrement, duplicate suppression, seeded loss, forwarding delay, expiry, packet traces. Uses no Spring and no keys.
Live simulator service	`MeshSimulatorService`	Owns the live network that the dashboard drives; performs the bridge upload step (parallel) and applies the backend answers to the devices.
Bridge	a `VirtualDevice` with role BRIDGE	Stores packets like any relay and, when online and in the mesh, submits them. It only ever handles opaque ciphertext.
Backend ingestion	`BridgeIngestionService`	The ordered pipeline for one delivery and its outcome policy.
Idempotency	`PacketClaimService`, table `packet_records`	Atomic first-delivery-wins claim: in-memory `putIfAbsent` plus a database UNIQUE row.
Encryption	`HybridCryptoService`, `ServerKeyHolder`, `Hashing`	Hybrid encryption, authenticated decryption, SHA-256 fingerprints, in-memory RSA key pair.
Settlement	`SettlementService`	The only code that changes balances: lock, check, debit, credit, ledger insert, mark settled, in one transaction.
Database and ledger	entities in `model`, H2	accounts, packet_records, ledger_entries, ingest_events.
Experiments	`ExperimentService`	Seeded parameter sweeps on private mesh instances that call the real backend pipeline.
REST layer	`controller.*`, `GlobalExceptionHandler`	Thin controllers, DTO records, one JSON error shape.
Frontend	`static/index.html`, `app.css`, `app.js`	Vanilla JavaScript dashboard; every number comes from a backend response.

2.3 Who trusts whom

Party	Trusted to...	Not trusted to...
Backend	Hold the private key, decide settlement, change balances.	Assume any packet is valid: it re-checks everything.
Sender device	Know its own plaintext payment.	Be honest about the amount or the time: the backend validates both.
Relay devices	Store and forward bytes.	Read the payment, keep packets honest, or deliver them at all.
Bridge devices	Upload bytes when online.	Decrypt (they cannot), settle, or report a result: only the backend's answer counts.
Dashboard (browser)	Display state.	Be a source of truth: balances and statuses shown are copies of backend responses.

2.4 Data flow in one sentence

Sender encrypts, injects at its phone, gossip copies the opaque packet across neighbours, a bridge with internet uploads it, the backend hashes, claims, decrypts, validates and settles it once, and the answer travels back to the bridge and the dashboard.

3. Technology stack

Versions below were read from the built jar (`BOOT-INF/lib`) and the project `pom.xml`. Only technologies that the code actually uses are listed.

Technology	Version	Purpose in this project
Java (language level)	17 (`<java.version>17</java.version>`, compiled with `--release 17`)	Records, switch expressions, `HexFormat`, text blocks. The final verification ran on OpenJDK 26.0.1.
Spring Boot	3.3.5	Application bootstrap, embedded Tomcat, auto-configuration, configuration properties binding, test support.
Spring Framework (web, tx, orm)	6.1.14	REST controllers, validation integration, declarative and programmatic transactions.
Spring Data JPA / Hibernate ORM	3.3.5 / 6.5.3.Final	Entities, repositories, pessimistic locking, optimistic versioning, unique-constraint mapping.
H2 database	2.2.224	Self-contained relational database (in-memory by default; file mode optional).
HikariCP	5.1.0	JDBC connection pool.
Embedded Tomcat	10.1.31	HTTP server for API and static dashboard.
Jackson	2.17.2	JSON for REST bodies and for the encrypted payload.
Hibernate Validator / Jakarta Validation	8.0.1.Final / 3.0.2	Request validation annotations (`@NotBlank`, `@Positive`, `@Min`, ...).
JDK cryptography (JCA)	built in	AES/GCM/NoPadding, RSA/ECB/OAEPWithSHA-256AndMGF1Padding, SHA-256, SecureRandom.
Maven (via wrapper)	3.9.9	Build and test. `./mvnw` downloads Maven on first use.
JUnit Jupiter, Spring Boot Test, MockMvc	5.10.5 / 3.3.5	Unit, integration and API tests.
Frontend	plain HTML, CSS, JavaScript (ES2020)	No framework, no bundler, no CDN. Works with the network unplugged.

> Not used: Thymeleaf, Kafka, Redis, Docker, any external database, any JavaScript framework, any web font download. The earlier reference projects used Thymeleaf only as a file server for the dashboard, so it was removed.

4. Installation and prerequisites

4.1 Required software

Tool	Requirement	How to check
Java JDK	17 or newer. The code is compiled for Java 17 and was verified on 26.0.1.	`java -version`
Maven	Not installed separately. The Maven Wrapper (`./mvnw`) downloads Maven 3.9.9.	`./mvnw -v` (run inside the project folder)
Internet	Needed **once**, so the wrapper can download Maven and the Maven dependencies. After that the whole demo runs offline.	-
Node.js	**Not required.** There is no frontend build.	-
A browser	Any current Chrome, Edge, Firefox or Safari (the page uses CSS `color-mix` and `overflow: clip`).	-

4.2 Configuration

All settings are in `src/main/resources/application.properties`. Each can also be overridden on the command line, for example `--upi.mesh.default-ttl=3`.

Property	Default	Meaning
`server.port`	`${PORT:8080}`	HTTP port; the `PORT` environment variable overrides it.
`spring.datasource.url`	`jdbc:h2:mem:upimesh;DB_CLOSE_DELAY=-1;LOCK_TIMEOUT=10000`	In-memory H2. `LOCK_TIMEOUT` is how long a settlement waits for a row lock.
`spring.jpa.hibernate.ddl-auto`	`create-drop`	Schema is created at start and dropped at stop.
`spring.web.resources.cache.cachecontrol.no-cache`	`true`	Browser must revalidate dashboard files, so an updated page is never served stale.
`upi.mesh.freshness-window-seconds`	86400	Replay window: older `signedAt` is rejected.
`upi.mesh.clock-skew-seconds`	300	Tolerated future-dated `signedAt`.
`upi.mesh.default-ttl`	5	Hop budget for new packets.
`upi.mesh.max-amount`	1000000000	Largest accepted payment.
`upi.mesh.idempotency-ttl-seconds`	86400	How long the in-memory fast path remembers a packet hash (the database is authoritative).
`upi.mesh.settlement-retries`	3	Attempts for transient settlement conflicts.
`upi.mesh.processing-timeout-seconds`	120	A claim stuck in PROCESSING longer than this is released.

Property	Default	Meaning
`upi.mesh.sim.*`	loss 0.0, delay 0, expiry 86400 s, seed 42	Mesh baseline restored by reset.
`upi.mesh.accounts[n].*`	alice 5000, bob 1000, carol 2500, dave 500	Fictional demo accounts and opening balances.

4.3 Keys and secrets

No key or secret exists in the source tree or in the documentation. `ServerKeyHolder` generates a fresh RSA-2048 key pair in memory every time the application starts. The private key is held in a package-private getter that only `HybridCryptoService` can reach, and it is never returned by any API, logged or sent to the browser. The public key is available from `GET /api/server-key`. Because the key is regenerated at every start, packets created in one run cannot be decrypted after a restart.

5. Complete execution guide

> Copy one command at a time and do not add trailing "# comments". Interactive zsh does not treat # as a comment, so Maven would receive the comment text as arguments and fail with "No plugin found for prefix 'http'".

5.1 Step by step

Step 1. Enter the project directory (the folder name "UPI mesh" contains a space, so keep the quotes):

```
cd "/Users/devxnsh/Desktop/code/UPI mesh/UPI-Offline-Mesh"
```

Step 2. Check prerequisites:

```
java -version
```

```
./mvnw -v
```

Step 3. Build the backend (this also packages the dashboard files):

```
./mvnw clean package -DskipTests
```

The jar is written to `target.nosync/upi-offline-mesh-1.0.0.jar`. The build folder is named `target.nosync` because macOS iCloud (Desktop and Documents sync) otherwise corrupts build output with files such as `Foo 2.class`; iCloud ignores names ending in `.nosync`. On other systems it is just a folder name.

Step 4. Run the backend tests:

```
./mvnw test
```

Step 5. Build the frontend: not applicable. The dashboard is static files served by Spring Boot; there is no separate frontend build and no second terminal.

Step 6. Start the complete application:

```
./mvnw spring-boot:run
```

Alternatively run the jar built in Step 3:

```
java -jar target.nosync/upi-offline-mesh-1.0.0.jar
```

Step 7. Local URL: http://localhost:8080 (set the `PORT` environment variable for another port, for example `PORT=9090 ./mvnw spring-boot:run`). On Windows use `mvnw.cmd` instead of `./mvnw`.

Step 8. Verify the backend is healthy (in a second terminal):

```
curl -s localhost:8080/api/health
```

Expected: `{"status":"UP","database":"UP","devices":8,"uptimeSeconds":...}`.

Step 9. Open the dashboard: browse to http://localhost:8080. The pill at the top right must say "BACKEND LIVE".

Step 10. Stop the application: press Ctrl+C in the terminal that is running it. The in-memory database and the mesh disappear with the process; the next start is the clean baseline.

5.2 Optional: keep data across restarts

Start with a file-backed H2 database. Accounts are seeded only when the database is empty, so balances, ledger and idempotency claims survive a restart (this was verified manually: settle a payment, stop, start again, balances and ledger row were still there).

```
./mvnw spring-boot:run -Dspring-boot.run.arguments="--spring.datasource.url=jdbc:h2:file:./data/upimesh;LOCK_TIMEOUT=10000 --spring.jpa.hiber
```

The mesh itself (devices and packets in flight) and the RSA key are always in memory, so packets from a previous run cannot be re-uploaded after a restart.

5.3 Quick troubleshooting

Symptom	Cause and fix
`Port 8080 was already in use`	Another process owns the port. Run with `PORT=9090 ./mvnw spring-boot:run`, or stop the other process.

Symptom	Cause and fix
`permission denied: ./mvnw`	Run `sh mvnw ...` or `chmod +x mvnw`.
Maven cannot download dependencies	Check internet access once; the wrapper needs to fetch Maven and libraries on first use. Retry the command.
`No plugin found for prefix 'http'` or unknown phase `#`	A trailing `# comment` was pasted with the command. Paste the command alone.
Tests find 0 tests or say `wrong name: ...Test 2`	iCloud conflict copies in the build folder. Run `./mvnw clean test`; the pom already builds into `target.nosync`.
Dashboard says "backend unreachable"	The application stopped. Start it again; the page retries every 2 seconds.
Database initialisation error	With a file database, delete the `data` folder and retry; with the default in-memory database restart the application.
Decryption always fails after a restart	The RSA key is regenerated at every start; create a new payment instead of re-uploading an old packet.

6. How to use the dashboard

The page is one scrolling document with a floating navigation pill at the bottom (Overview, Mesh, Packets, Settlement, Prove it, Log). All data refreshes from the backend every two seconds and after every action.

6.1 Overview (hero)

Control or indicator	Purpose and behaviour	Backend source
Clock	Shows your local time; decorative.	Browser clock
"Simulation - fictional accounts" pill	Reminds the viewer that nothing is real.	-
"Backend live" pill	Green while polling succeeds; red with a banner when the backend cannot be reached.	All polling requests
Start a payment	Scrolls to the payment form.	None (anchor)
Run all scenarios	Scrolls to "Prove it" and runs the 8 guided scenarios.	Many (see 6.6)
Eight statistic tiles	Simulated devices, devices in the mesh, bridges with internet, packets awaiting delivery, packets forwarded, settlements, duplicates dropped, invalid rejected. A tile flashes when its value changes.	`GET /api/stats`

6.2 Mesh topology

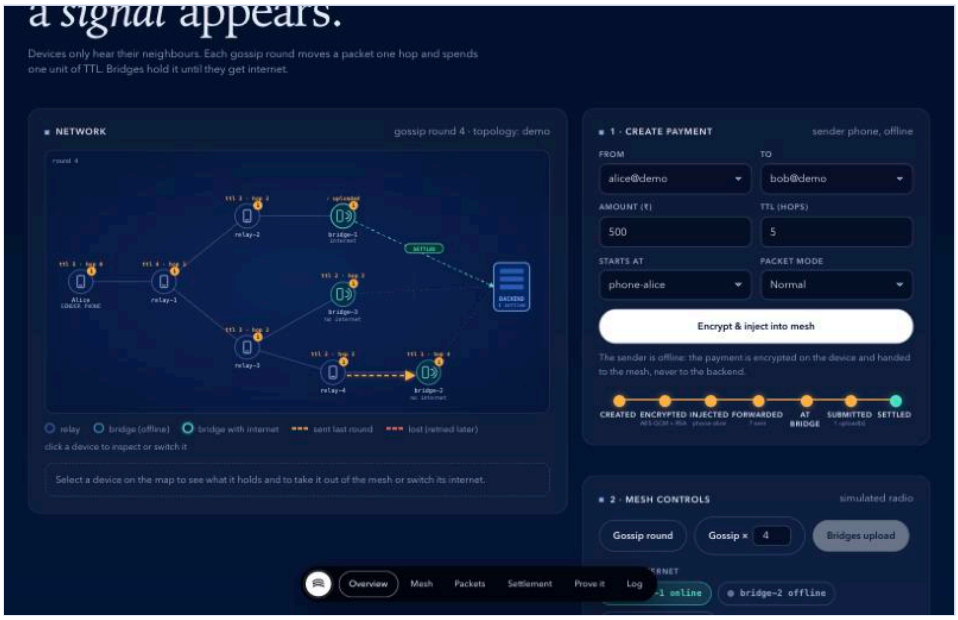


Figure 6.1: Topology after a payment settled. Amber badges show packet copies, labels show TTL and hop count, the green badge on the bridge link shows the backend outcome.

- **Nodes.** Circles are phones. A bridge has a signal icon. A teal ring with a ripple means the bridge has internet. A dashed, faded node is out of the mesh. The small amber number is how many packet copies the device holds.
- **Above a node:** `ttl N – hop H` for the selected packet (remaining hops and hops travelled), `uploaded` when the backend acknowledged that copy, or `expired`.
- **Lines.** Solid grey lines are neighbour links (dashed when an end is out of the mesh). Dashed amber arrows are transmissions sent in the most recent round; dashed red arrows are lost transmissions that will be retried. Teal dashed lines from a bridge to BACKEND appear only while that bridge has internet.
- **Badges on the internet links** (SETTLED, DUPLICATE, REJECTED, FAILED) are the real upload outcomes recorded for the selected packet.
- **Clicking a device** opens the strip below the map: it shows role, neighbours and copies held, and offers "Take out of mesh / Bring back" (`PATCH /api/mesh/devices/{id}` with `active`) and, for bridges, "Give / Cut internet" (`online`).

6.3 Create payment

Control	Purpose	Backend operation	What you should see	Failure looks like
From / To	Choose sender and receiver (fictional accounts).	None until submit	Submit is disabled if they are equal, with an explanation.	Disabled button and hint text

Control	Purpose	Backend operation	What you should see	Failure looks like
Amount	Rupees, up to two decimals.	Validated by `@Positive` `@Digits`	Submit disabled for 0 or blank.	Toast with the validation message
TTL (hops)	Hop budget of this packet (1 to 32).	Sent as `ttl`	Defaults to the mesh setting.	Toast: validation failed
Starts at	Device that receives the packet (default: the sender phone).	Sent as `startDevice`	Packet appears on that node.	Toast: unknown device (400) or device not in mesh (409)
Packet mode	Normal; Duplicate (copy on every bridge); Tampered (one ciphertext byte flipped); Expired (signed outside the freshness window).	Sent as `mode`	The history shows an ATTACK event for the last three.	-
Encrypt & inject into mesh	Creates, encrypts and injects the packet. **No settlement endpoint is called.**	`POST /api/demo/send`	Toast "Packet ... encrypted and injected"; Alice shows `ttl 5 – hop 0`; stages Created, Encrypted, Injected light up; Awaiting delivery becomes 1.	Red toast with the server message

The **stage tracker** under the form is computed from the selected packet's real event history: Created, Encrypted, Injected, Forwarded (n sent), At bridge, Submitted (n uploads), then the final outcome (SETTLED, REJECTED or DUPLICATE). A stage lights up only when the corresponding event exists. A packet that is merely forwarded is never shown as settled.

6.4 Mesh controls

Control	Purpose	Backend operation	What you should see	Why it may be disabled or fail
Gossip round	One hop of spreading.	`POST /api/mesh/gossip {rounds:1}`	Arrows on the map; toast "n sent, m lost, k suppressed".	Disabled until a packet exists (tooltip explains)
Gossip x N	N rounds in one call.	`POST /api/mesh/gossip {rounds:N}`	Same, aggregated.	Same; N must be 1 to 50
Bridges upload	Every online bridge uploads every un-acknowledged packet it holds, simultaneously.	`POST /api/mesh/flush`	Outcome badges on the internet links; ledger rows; toast "n upload(s): ...".	Disabled when no bridge has internet or none holds anything (hint text says which)
Bridge toggles	Give or cut a bridge's internet.	`PATCH /api/mesh/devices/{id} {online}`	Teal ripple appears or disappears.	Disabled if the bridge is out of the mesh
Default TTL, Loss, Delay, Expiry, Seed	Tune the simulated network.	`PUT /api/mesh/settings` on change	Values persist until reset.	Out-of-range values return 400
Topology	demo (3 bridges), line (5 hops) or full mesh (broadcast).	`POST /api/mesh/reset {topology}`	Map redraws; everything resets.	-
Reset to baseline	Restores accounts, ledger, idempotency claims, mesh and the sender outbox. Two clicks are required (the button asks "Click again to confirm").	`POST /api/mesh/reset`	Balances back to 5000/1000/2500/500; ledger empty; map clean.	-

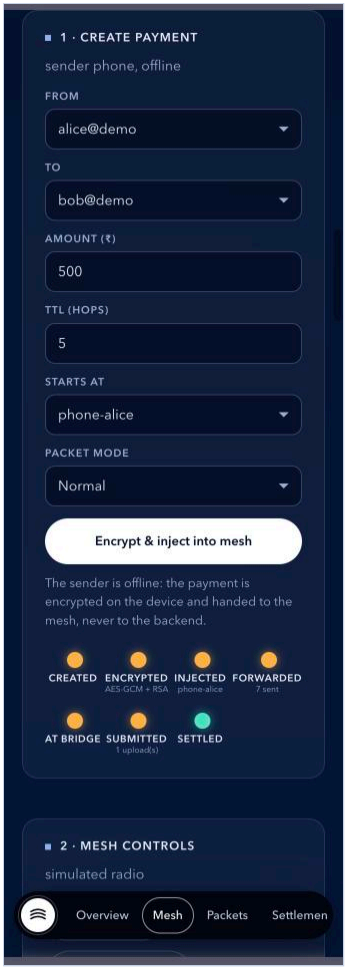


Figure 6.2: The payment form and stage tracker on a phone-width screen after a settled payment.

6.5 Packet inspection

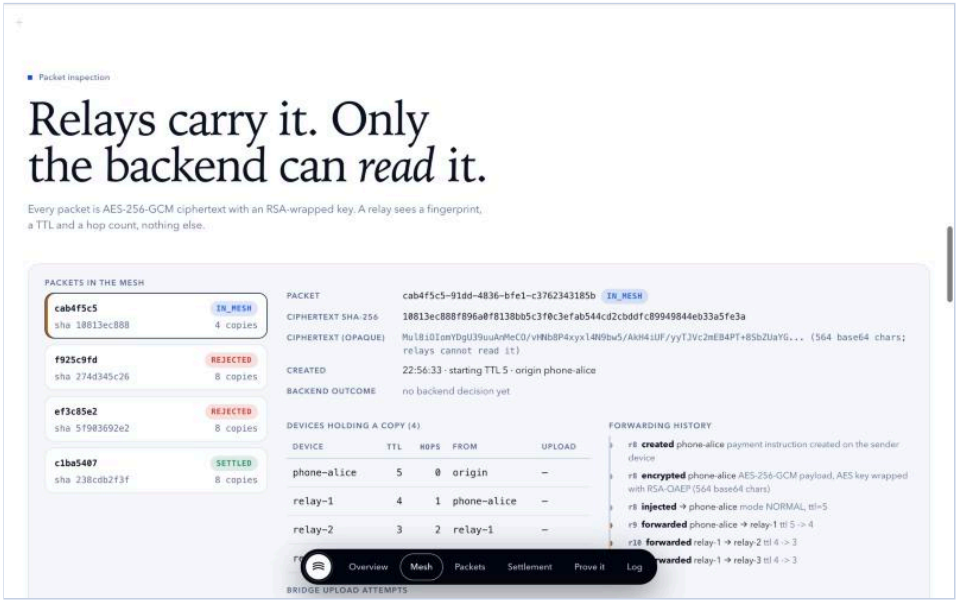


Figure 6.3: Packet list, fingerprint, holders, forwarding history and attack buttons.

- **Packets in the mesh** lists packets newest first with status (IN_MESH, AT_BRIDGE, SETTLED, REJECTED, DUPLICATE, TTL_EXHAUSTED, EXPIRED). Click one to inspect it (`GET /api/mesh/packets/{id}`).
- **Ciphertext SHA-256** is the packet fingerprint. **Ciphertext (opaque)** is a truncated Base64 preview, explicitly labelled unreadable. The page never shows plaintext or key material.
- **Devices holding a copy** shows TTL, hops, who it came from and the upload state per device.
- **Backend outcome** lists every backend decision for this packet with its reason and ledger id; "no backend decision yet" while it is still travelling.

- **Forwarding history** is the chronological event list for the packet (created, encrypted, injected, forwarded, lost, suppressed, bridge received, upload attempts, attacks, TTL and expiry events).
- **Replay attacks.** "Submit identical bytes" sends the exact bytes to the backend N times at once (`POST /api/demo/replay` kind IDENTICAL). "Repackage same payment" re-encrypts the same payment (same nonce, new ciphertext) as a new packet injected at the origin (kind REPACKAGED); gossip and upload it to see the backend reject it as NONCE_REUSED.

6.6 Ledger, accounts, scenarios, experiments, log

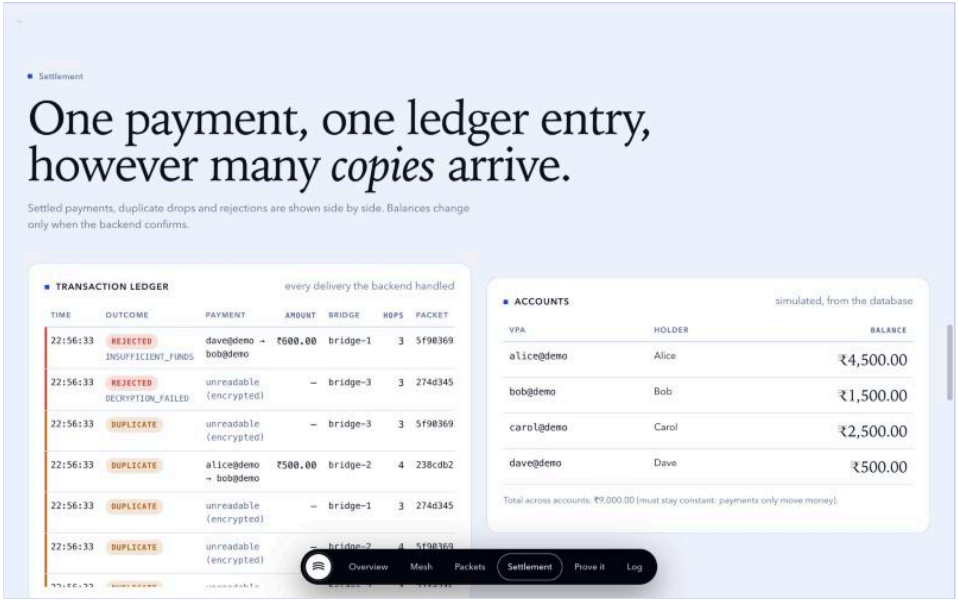


Figure 6.4: Ledger rows for settled, duplicate and rejected deliveries, and the account balances.

- **Transaction ledger** (`GET /api/transactions`, newest 50): one row per delivery the backend handled. Outcome badge colours: teal SETTLED, amber DUPLICATE, red REJECTED. Payment details show "unreadable (encrypted)" when the backend could not decrypt the packet (for example a tampered one). A rejected or duplicate packet never looks like a settlement.
- **Accounts** (`GET /api/accounts`): balances from the database and the total across accounts, which must stay constant because payments only move money.
- **Guided scenarios**: eight buttons plus "Run all". Each one resets the demo, drives the same endpoints as the manual controls, and prints a check mark or a red failure line for each assertion against live balances, ledger and packet status (27 checks in total).
- **Networking experiments**: pick an experiment, edit the values, runs per value and seed, press "Run experiment" (`POST /api/experiments/run`). A line chart, a table and the reproduction parameters are shown. "Metric" switches the plotted quantity; "Earlier runs" reloads previous results.
- **Activity log**: the most recent mesh events from the backend, newest first.



Figure 6.5: Guided scenario log (phone width).



Figure 6.6: Experiment chart and table (phone width).

6.7 Step by step: the normal payment

1. Press "Reset to baseline" twice. Confirm the four balances (5000, 1000, 2500, 500).
2. In "Create payment" choose From = alice@demo, To = bob@demo, Amount = 500 and press "Encrypt & inject into mesh". Alice's node shows `ttl 5 – hop 0`; the first three stages light up. The ledger and balances do not change.
3. Press "Gossip x 4". The packet spreads one hop per round; the Forwarded and At bridge stages light up. All three bridges now hold a copy. Balances are still unchanged.
4. Press "Bridges upload". It is disabled with the hint "No bridge has internet", because every bridge is offline. Nothing can settle.
5. Click the `bridge-1` toggle so it goes online (teal ripple). Press "Bridges upload". Only bridge-1 uploads: the toast says "1 upload(s): 1 settled", a SETTLED badge appears on its internet link, Alice is debited to 4,500 and Bob credited to 1,500, and one SETTLED row appears in the ledger.
6. Switch `bridge-2` and `bridge-3` online and upload again. They still hold identical copies, so the backend answers DUPLICATE twice; balances and ledger do not change.

6.8 Step by step: the security and network demonstrations

Demonstration	Do this (after Reset)	Expected result
Duplicate delivery	Mode = Duplicate, send; switch all 3 bridges online; Bridges upload once.	3 concurrent uploads: 1 SETTLED, 2 DUPLICATE. Then in Packet inspection submit identical bytes x6: 6 DUPLICATE. Balances moved once.
Repackaged duplicate	After a settled payment press "Repackage same payment", gossip x4, bridges online, upload.	The new packet is REJECTED with NONCE_REUSED. The sender is debited once.
Tampered ciphertext	Mode = Tampered, gossip x4, one bridge online, upload.	REJECTED, DECRYPTION_FAILED (a real AES-GCM tag failure). No balance or ledger change.
Expired packet	Mode = Expired, same steps.	REJECTED, STALE_TIMESTAMP. Balances unchanged.
TTL exhaustion	TTL = 2, gossip x6, all bridges online.	Only phone-alice, relay-1, relay-2, relay-3 hold the packet; status TTL_EXHAUSTED; upload has nothing to send.
Interrupted connectivity	Send, gossip x8 with every bridge offline, then switch bridge-2 online and upload.	Nothing settles while offline; afterwards the same packet settles without the sender doing anything.
Insufficient funds	dave@demo to bob@demo, 600 (Dave has 500), gossip x4, bridges online, upload.	REJECTED, INSUFFICIENT_FUNDS. No partial update, no SETTLED row.

Demonstration	Do this (after Reset)	Expected result
Concurrent updates	Guided scenario 8.	36 simultaneous uploads: 12 SETTLED, 24 DUPLICATE; exact balances; total money unchanged.

7. Payment lifecycle

This chapter follows one fictional payment: **Alice sends Bob 500 rupees** while Alice has no internet. The identifiers are taken from a real run of the application.

7.1 The journey

Stage	What happens	Where it is recorded
1. Create	`DemoService` builds `PaymentInstruction{senderVpa: alice@demo, receiverVpa: bob@demo, amount: 500, nonce: random UUID, signedAt: now}`.	Sender outbox (memory). Not visible to relays.
2. Encrypt	A new AES-256 key and 12-byte IV encrypt the JSON; the key is wrapped with RSA-OAEP; the result is one Base64 blob (about 560 characters). Alice never calls a settlement endpoint.	Event ENCRYPTED
3. Inject	A `MeshPacket{packetId 4888dbde-..., ttl 5, createdAt, ciphertext}` is stored on `phone-alice`.	Event INJECTED; mesh store of phone-alice
4. Gossip	Round 1: copy to relay-1 (ttl 4). Round 2: relay-2 and relay-3 (ttl 3). Round 3: bridge-1, bridge-3, relay-4 (ttl 2). Round 4: bridge-2 (ttl 1).	Events FORWARDED; copies on each device
5. Deferred storage	Bridges hold their copy even though they are offline. The packet status is AT_BRIDGE. Nothing is settled: Alice still has 5000.	Event BRIDGE_RECEIVED
6. Bridge comes online	bridge-1 gets internet. "Bridges upload" collects its un-acknowledged copy.	Device flag `online`
7. Ingestion	The backend computes SHA-256 of the ciphertext (fingerprint 66c2eba0...), claims it, decrypts it with the private key, checks fields and freshness.	Row in `packet_records` (PROCESSING)
8. Settlement	One transaction: lock Alice and Bob, check balance, debit 500, credit 500, insert the ledger entry, mark the packet SETTLED.	`accounts`, `ledger_entries`, `packet_records`
9. Acknowledgement	The bridge receives SETTLED and marks its copy ACKED; the audit row is written.	`ingest_events`; upload attempt in the packet trace
10. Later copies	bridge-2 and bridge-3 come online and upload identical bytes: the claim already exists, the answer is DUPLICATE_DROPPED and nothing changes.	`ingest_events` (DUPLICATE_DROPPED)

7.2 Sequence diagram

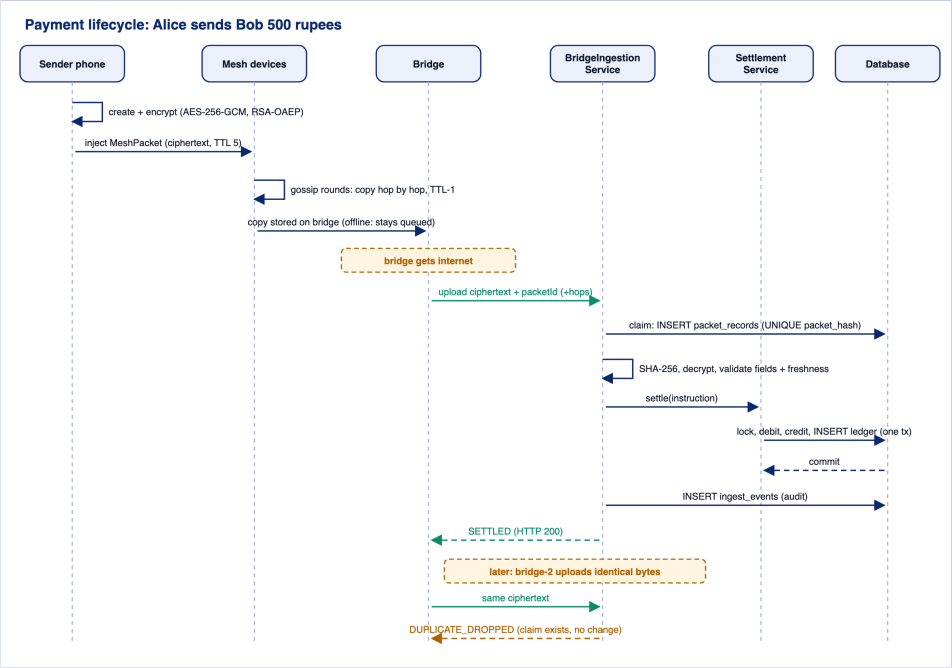


Figure 7.1: Sequence of the payment lifecycle, including the later duplicate upload.

7.3 Delivery, ingestion and settlement are different things

Term	Meaning	Example evidence
Packet delivery	A copy reached a device (relay or bridge) in the mesh.	The device appears in "Devices holding a copy".

Term	Meaning	Example evidence
Backend ingestion	The backend received a copy and made a decision about it (any of the four outcomes).	A row in `ingest_events`; a badge on the internet link.
Financial settlement	Balances changed and a ledger entry exists.	A `ledger_entries` row and new balances; outcome SETTLED.

A packet can be delivered to all eight devices and never be ingested (all bridges offline). It can be ingested and rejected (tampered). Only SETTLED means money moved. The dashboard therefore tracks the three stages separately.

7.4 When the bridge stays offline

The packet remains at the bridge with upload state PENDING and the status AT_BRIDGE. Gossip rounds change nothing further. "Bridges upload" is disabled with an explanation. Nothing is lost: the copy persists until a bridge with internet uploads it, or until the mesh expiry policy (24 hours of outer age by default) makes devices stop forwarding it. When connectivity returns, the next upload submits the same bytes, and the sender does nothing.

8. Networking concepts

Concept	How this project demonstrates it	Where in the code	Limitation of the simulation
Mesh topology	Devices only communicate with direct neighbours. In the demo topology Alice reaches only relay-1; bridge-1 and bridge-3 are 3 hops away and bridge-2 is 4 hops away.	`Topology.demo()`	Static topology; devices do not move.
Neighbour relationships	Each device has a sorted neighbour set; gossip only offers packets to these.	`Topology.neighbors`	Links are on or off, with no signal strength or range model.
Store and forward	A device keeps a packet until it can forward it. A failed forward keeps the sender's copy.	`MeshNetwork.gossipRound`, `StoredPacket`	Storage is unlimited; no queue eviction.
Gossip propagation	Each round, each device offers each held packet to each neighbour lacking it; exactly one hop per round (start-of-round snapshot).	`gossipRound()`	Deterministic order (by device id), not random pairing.
Packet replication	Every neighbour gets its own copy; the number of copies is visible per packet.	`StoredPacket`, packet detail	Replication is flooding with TTL, not a smarter protocol.
TTL	The receiver's copy has TTL-1. At 0 a copy stays but is not forwarded; a bridge with TTL 0 can still upload.	`gossipRound`, `status()`	TTL counts hops, not time.
Duplicate suppression	A neighbour that already holds, or is already receiving, a packet is skipped and counted as "suppressed".	`gossipRound (suppressed)`	Models an inventory check; real devices would need a handshake.
Intermittent connectivity	Devices can leave the mesh; bridges can lose internet; both are toggles.	`setActive`, `setOnline`	State changes by button or API, not by mobility.
Bridge (gateway) connectivity	Only a BRIDGE that is in the mesh and online can upload.	`pendingUploads()`	Binary online/offline, no bandwidth or latency.
Delivery delay	`delayRounds` keeps a transmission in flight for extra rounds.	`Arrival`, `deliverDueArrivals`	Delay is in rounds, not milliseconds.
Packet loss	Each transmission is lost with probability p using a seeded random generator; the sender retries next round.	`gossipRound (lost)`	Independent, identically distributed loss only.
Duplicate delivery	Several bridges hold identical copies and upload them together.	`MeshSimulatorService.flush`	The race is real (threads and database), the bridges are simulated.
Network reliability vs application guarantees	The mesh gives best-effort, at-least-once, possibly duplicated delivery. The backend adds idempotent settlement so the application effect happens once.	`BridgeIngestionService`, `PacketClaimService`	-

8.1 The gossip algorithm in the source

```
gossipRound():
  round++ ; mark copies older than packetExpirySeconds as expired
  snapshot = copies held by every device at the START of the round     // one hop per round
  for device d in id order (if active):
    for copy c in snapshot[d] (skip expired):
      if c.ttl == 0: log TTL_EXHAUSTED if a neighbour still lacks the packet; continue
      for neighbour n of d in id order (if active and not already served by c):
        if n holds the packet or is already receiving it:  suppressed++ ; continue
        if random() < lossProbability:                      lost++ ; continue   // copy kept, retried next round
        schedule n.store(copy with ttl-1, hops+1) at round + delayRounds ; sent++
  deliver arrivals that are due
```

8.2 Network reliability versus settlement guarantees

The mesh can deliver a packet zero times (partition, TTL exhaustion, expiry), once, or many times. Nothing in the mesh can promise "exactly once". The guarantee that matters to the user, one ledger effect per logical payment, is provided at the application layer by three mechanisms working together: the ciphertext-hash claim, the unique nonce in the ledger, and the single settlement transaction (Chapter 9 and Chapter 10). This separation is the main networking lesson of the project: the transport may be unreliable and duplicating as long as the receiver is idempotent.

9. Cryptography and security

9.1 Hybrid encryption

Public-key encryption (RSA) can only encrypt small data and is slow. Symmetric encryption (AES) is fast and handles any size but needs a shared key. Hybrid encryption combines them: AES encrypts the payload with a fresh random key, and RSA encrypts only that small key for the server.

```
sender:  K  = fresh AES-256 key (KeyGenerator)          IV = fresh 12 random bytes (SecureRandom)
         C || tag = AES/GCM/NoPadding(K, IV, json(PaymentInstruction))        // 128-bit tag
         W  = RSA/ECB/OAEPWithSHA-256AndMGF1Padding(serverPublicKey, K)       // 256 bytes for RSA-2048
         blob = Base64( W[256] || IV[12] || C || tag[16] )
backend: K = RSA-OAEP-decrypt(privateKey, W) ; plaintext = AES-GCM-decrypt(K, IV, C || tag)
```

- **AES-256-GCM** gives confidentiality and integrity. GCM computes an authentication tag over the ciphertext; if any single bit of the blob changes, decryption throws `AEADBadTagException`, which the service reports as `DECRYPTION_FAILED`. That is how tampering is detected.
- **RSA-OAEP** protects the AES key. The code uses `OAEPParameterSpec("SHA-256", "MGF1", MGF1ParameterSpec.SHA256, PSource.PSpecified.DEFAULT)`: SHA-256 for both the OAEP digest and the MGF1 mask function (explicitly configured, not left to provider defaults).
- **A fresh key per packet** means the same IV can never be reused under the same key.
- **SHA-256 fingerprint** of the Base64 ciphertext text (`Hashing.sha256Hex`) identifies byte-identical packets and is what the dashboard shows. It is computed without any key, so relays can compute it too.
- **Binary values** are serialised as one standard Base64 string with a fixed layout, so the format is stable across devices.

9.2 What a relay can and cannot see

Data	Visible to a relay?	Authenticated?
`packetId`, `ttl`, `createdAt` (outer fields)	Yes (needed for forwarding and expiry)	**No.** A malicious relay may rewrite them, so the backend never trusts them.
Ciphertext blob	Yes, as opaque bytes	Yes: any change fails GCM authentication.
Sender, receiver, amount, nonce, `signedAt`	No (inside the ciphertext)	Yes (inside AES-GCM).
AES key	No (RSA-wrapped)	-
RSA private key	No (never leaves `ServerKeyHolder`; not logged, not in any API)	-

9.3 Replay protection, nonces and idempotency

Encryption does not prevent replay. An attacker who copies a valid encrypted packet can send it again; the backend would decrypt it successfully. Replay is handled by separate mechanisms:

Case	Mechanism	Outcome
1. Byte-identical packet again	Ciphertext hash claim (`packet_records`, UNIQUE `packet_hash`).	DUPLICATE_DROPPED
2. Same logical payment, new ciphertext	Unique `nonce` in `ledger_entries`. Re-encrypting gives different bytes but the same nonce.	REJECTED, NONCE_REUSED
3. Two real payments, same parties and amount	Different random nonces, therefore different packets.	Both SETTLED
4. Packet signed too long ago	`signedAt` inside the authenticated payload is older than the freshness window (24 h).	REJECTED, STALE_TIMESTAMP
5. `signedAt` in the future beyond the skew	More than 300 s ahead of the backend clock.	REJECTED, FUTURE_TIMESTAMP

Why a ciphertext hash alone is insufficient. Encryption uses a fresh AES key and IV every time, so the same instruction encrypted twice produces two completely different ciphertexts with two different hashes. A hash-only check would accept both. The nonce lives inside the encrypted instruction and is identical in both, so a unique database constraint on it stops the second settlement. The test `case2_sameLogicalPaymentRepackagedUnderNewCiphertextCannotSettleTwice` demonstrates this.

Timestamp freshness is not a substitute for idempotency. A packet replayed inside the 24-hour window is still fresh; only the claim and the nonce stop it. Conversely idempotency does not remove the need for freshness, which bounds how long nonces must be remembered.

9.4 Security boundaries and honesty

- Relays can still **drop or delay** packets; cryptography does not prevent denial of delivery.
- The RSA key is regenerated at every start and exists only in memory; production would need an HSM or KMS, rotation and forward secrecy.
- There is no user authentication, PIN verification, bridge authentication, rate limiting or TLS. The demo controls (`/api/demo/*`, `/api/mesh/reset`) and the H2 console are open. **Do not expose this application to an untrusted network.**
- Logs contain only packet-hash prefixes, bridge ids, hop counts and ledger ids, never plaintext payments, keys or full ciphertext.
- An offline sender cannot know whether it can pay. A malicious sender may create several offline payments; only those the balance covers will settle. This is a fundamental limit of offline payments, not a coding gap.

10. Database and backend internals

10.1 Schema

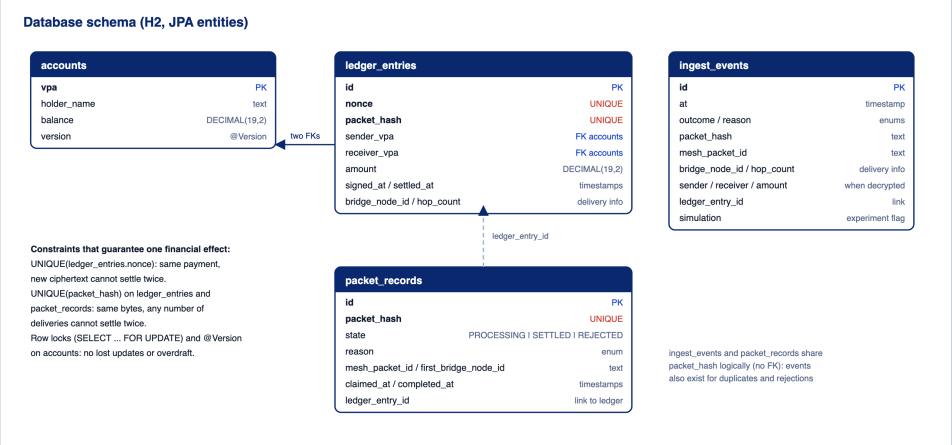


Figure 10.1: Database schema and the constraints that guarantee one financial effect.

Table / entity	Role	Key constraints
`accounts` / `Account`	Fictional bank accounts. `balance` is `BigDecimal` (scale 2).	PK `vpa`; `@Version` optimistic locking.
`ledger_entries` / `LedgerEntry`	One immutable row per successfully settled payment.	`uk_ledger_nonce`, `uk_ledger_packet_hash`; FKs to accounts.
`packet_records` / `PacketRecord`	Idempotency claim and processing state machine, one row per distinct ciphertext.	`uk_packet_records_hash` (UNIQUE `packet_hash`).
`ingest_events` / `IngestEvent`	Audit row for every delivery including duplicates and rejections. This is what the dashboard ledger shows. `simulation=true` marks experiment traffic.	None beyond PK.

10.2 Packet state machine and outcomes

`PacketRecord` moves PROCESSING to SETTLED or REJECTED (both final). A transient failure **deletes** the PROCESSING row so the packet can be retried, which guarantees a packet is never stuck in an ambiguous state. Outcomes returned to the bridge:

Outcome	Meaning	Retry?	HTTP at `/api/bridge/ingest`
SETTLED	First valid delivery; money moved.	No	200
DUPLICATE_DROPPED	Byte-identical packet already claimed or processed.	No	200
REJECTED + reason	Permanent: MALFORMED_PACKET, DECRYPTION_FAILED, MALFORMED_PAYLOAD, INVALID_AMOUNT, SAME_ACCOUNT, STALE_TIMESTAMP, FUTURE_TIMESTAMP, UNKNOWN_SENDER, UNKNOWN_RECEIVER, INSUFFICIENT_FUNDS, NONCE_REUSED.	No	422
FAILED + reason	Transient: CONCURRENCY_CONFLICT or PROCESSING_ERROR. The claim was released.	Yes	503 with Retry-After

10.3 Services and transaction boundaries

Class	Transaction behaviour	Why
`BridgeIngestionService`	Not transactional.	Each step owns its transaction, so a late failure can still release the claim.
`PacketClaimService.claim`	`TransactionTemplate`, `REQUIRES_NEW`; in-memory `putIfAbsent` first, then INSERT.	The claim must commit independently so that concurrent requests see it immediately.
`PacketClaimService.reject / release`	`REQUIRES_NEW`	Recording a rejection or releasing a claim must not depend on another transaction.
`SettlementService.settle`	`@Transactional` (one transaction)	Debit, credit, ledger insert and marking the packet SETTLED commit or roll back together.
`ResetService.reset`	`@Transactional`	Restores the baseline atomically.

10.4 Why balances and the ledger stay consistent

1. `settle()` first validates amount (positive, at most 2 decimals, within the cap), distinct parties and an unused nonce.
2. It locks both account rows with `SELECT ... FOR UPDATE` (`PESSIMISTIC_WRITE`) in alphabetical order, so two opposite transfers cannot deadlock and concurrent transfers on the same account are serialised.
3. It checks the balance after acquiring the lock, so concurrent spending cannot overdraw (test: ten concurrent payments of 100 from an account holding 500 settle exactly five).
4. It changes both balances and inserts the ledger row with `saveAndFlush`; a UNIQUE violation on nonce or packet hash aborts the whole transaction, including both balance updates (test: `failureAfterDebitRollsBackBalancesAndReleasesTheClaim`).
5. `Account` carries `@Version`, so a stray concurrent update would fail instead of overwriting.
6. Transient conflicts (lock timeout, optimistic-lock failure) are retried up to 3 times inside the backend; persistent ones become FAILED with the claim released.

> An in-memory cache or a Java `synchronized` block is deliberately **not** the safety mechanism. The cache is a fast path; the database constraints and row locks are authoritative (the test `databaseConstraintStillCatchesDuplicatesWhenTheInMemoryFastPathIsEmpty` clears the cache and shows the database still drops the duplicate).

10.5 Persistence and reset behaviour

Data	Where it lives	Survives restart?	Cleared by reset?
Accounts, ledger, packet records, audit events	H2 database	No with the default in-memory URL; yes with the optional file URL	Yes (accounts return to the configured balances)
Devices, packet copies, forwarding history, upload attempts	Memory (`MeshNetwork`)	No	Yes (new network)
In-memory idempotency fast path	Memory (`PacketClaimService`)	No (database still authoritative)	Yes
Sender outbox (plaintext of own packets)	Memory (`DemoService`)	No	Yes
RSA key pair	Memory (`ServerKeyHolder`)	No (regenerated)	No
Mesh settings (TTL, loss, delay, expiry, seed)	Memory	No	Yes (back to configuration)
Experiment history (last 20)	Memory (`ExperimentService`)	No	No (results are kept)

A reset deletes events, ledger rows and packet records (children first), deletes and re-seeds accounts from `upi.mesh.accounts`, clears the claim cache, creates a new mesh and clears the sender outbox. Experiments use two hidden accounts (`sim-sender@sim`, `sim-receiver@sim`) and bridge ids starting with `sim-`; their rows are flagged and never appear in the dashboard views or statistics.

11. API reference

Base URL `http://localhost:8080`. Bodies are JSON (`Content-Type: application/json`). Every error uses the same shape:

```
{"timestamp":"2026-10-09T17:35:57.764495Z","status":404,"error":"NOT_FOUND","message":"Unknown device: ghost","details":[]}
```

`error` is one of VALIDATION_FAILED, MALFORMED_REQUEST, BAD_REQUEST (400), NOT_FOUND (404), CONFLICT (409), INTERNAL_ERROR (500). Unknown routes and wrong HTTP methods return Spring's standard 404 and 405. Stack traces and internal database messages are never returned.

Method	Path	Purpose
GET	`/`	Dashboard (forwards to `index.html`)
GET	`/api/health`	Liveness: process, database, device count, uptime
GET	`/api/server-key`	Public key, fingerprint, algorithm
GET	`/api/accounts`	Demo account balances
GET	`/api/transactions`	Newest 50 deliveries (settled, duplicate, rejected, failed)
GET	`/api/stats`	Overview counters
GET	`/api/mesh/state`	Topology, devices, packets, last transmissions, events, counters
POST	`/api/demo/send`	Create, encrypt and inject a payment
POST	`/api/demo/replay`	Replay attacks on a packet
POST	`/api/mesh/gossip`	Run gossip rounds
POST	`/api/mesh/flush`	Online bridges upload
POST	`/api/mesh/reset`	Restore the baseline
PATCH	`/api/mesh/devices/{id}`	Switch a device in/out of the mesh or a bridge online/offline
GET / PUT	`/api/mesh/settings`	Read / update simulation settings
GET	`/api/mesh/packets/{packetId}`	Packet inspection
POST	`/api/bridge/ingest`	Production-shaped ingestion endpoint
GET	`/api/experiments/types` and `/api/experiments`	Experiment catalogue; recent results
POST	`/api/experiments/run`	Run an experiment sweep

11.1 GET /api/health

No parameters. Example response: `{"status":"UP","database":"UP","devices":8,"uptimeSeconds":4}`. `status` is `DEGRADED` and `database` is `DOWN` if the database cannot be queried.

11.2 GET /api/server-key

```
{"publicKey":"MIIBIjANBgkq...","fingerprint":"<64 hex>","algorithm":"RSA-2048 / OAEP-SHA256","hybridScheme":"RSA-OAEP wraps a fresh AES-256-C
```

Returns the X.509 public key only. The private key is never exposed.

11.3 GET /api/accounts and GET /api/transactions

```
[{"vpa":"alice@demo","holderName":"Alice","balance":4500.00}, ...]
```

```
[{"id":1,"at":"2026-10-09T14:47:45.057668Z","outcome":"SETTLED","reason":null,"senderVpa":"alice@demo","receiverVpa":"bob@demo","amount":500.
```

Transactions are newest first. For rejections that could not be decrypted, `senderVpa`, `receiverVpa` and `amount` are null. Experiment traffic is excluded.

11.4 GET /api/stats

```
{"devicesTotal":8,"devicesActive":8,"bridgesTotal":3,"bridgesOnline":1,"packetsAwaiting":0,"packetsForwarded":7,"transmissionsLost":0,"duplic
```

11.5 GET /api/mesh/state

Top-level keys: `round`, `settings`, `topology`, `senderDevice`, `devices`, `links`, `packets`, `lastTransfers`, `counters`, `events`. A device looks like `{"id":"bridge-1","label":"bridge-1","role":"BRIDGE","x":590,"y":130,"online":false,"active":true,"neighbors":`

`["relay-2"],"copies":[]}`; a packet like `{"packetId":"...","status":"IN_MESH","fingerprint":"...","createdAt":...,"initialTtl":5,"origin":"phone-alice","holders":["phone-alice","relay-1"]}`; a last-round transfer like `{"round":1,"from":"phone-alice","to":"relay-1","packetId":"...","kind":"SENT"}` (`kind` is SENT or LOST).

11.6 POST /api/demo/send

Field	Required	Rules
`senderVpa`	yes	Not blank
`receiverVpa`	yes	Not blank
`amount`	yes	Greater than 0; at most 12 integer digits and 2 decimals
`ttl`	no	1 to 32; default from settings
`startDevice`	no	Device id; default is the sender phone
`mode`	no	NORMAL (default), DUPLICATE, TAMPERED, EXPIRED

```
POST /api/demo/send
{"senderVpa":"alice@demo","receiverVpa":"bob@demo","amount":500}
```

```
{"packetId":"ee7cf341-58b2-4555-af1a-af8b401e240b","mode":"NORMAL","ttl":5,"injectedAt":"phone-alice","replicatedTo":[],"fingerprint":"373d0b
```

Failures: 400 VALIDATION_FAILED (for example `"amount: must be greater than 0"`), 400 MALFORMED_REQUEST (bad JSON), 400 BAD_REQUEST (unknown `startDevice`), 409 CONFLICT (start device is out of the mesh). The response never contains the sender, receiver or amount.

11.7 POST /api/demo/replay

Body `{"packetId":"...","kind":"IDENTICAL|REPACKAGED","times":1-50}` (`times` optional, used by IDENTICAL).

```
{"kind":"IDENTICAL","submissions":2,"summary":{"SETTLED":1,"DUPLICATE_DROPPED":1},"results":[{"outcome":"SETTLED","reason":null,"packetHash":
```

For REPACKAGED the response has `results: []` and `injected` holds the new packet (same shape as the send response). Failures: 404 unknown packet, 409 if the sender no longer remembers the payment (after a reset).

11.8 POST /api/mesh/gossip, /flush, /reset

```
POST /api/mesh/gossip   {"rounds":4}     (body optional; rounds 1-50, default 1)
-> {"rounds":4,"transfers":7,"lost":0,"suppressed":0,"expired":0,"perRound":[{"round":1,"transfers":1,...}, ...]}
```

```
POST /api/mesh/flush
-> {"uploadsAttempted":1,"summary":{"SETTLED":1},"results":[{"bridgeNode":"bridge-1","packetId":"...","outcome":"SETTLED","reason":null,"ledg
```

```
POST /api/mesh/reset   {"topology":"demo|line|full-mesh"}   (body optional)
-> {"status":"reset","topology":"demo"}
```

Failures: gossip `rounds` outside 1 to 50 returns 400; reset with an unknown topology returns 400.

11.9 PATCH /api/mesh/devices/{id}

Body `{"online":true|false,"active":true|false}`; either field may be omitted. Returns the updated device. 404 for an unknown device; 400 when setting `online` on a relay (only bridges can have internet).

11.10 GET / PUT /api/mesh/settings

```
{"defaultTtl":5,"lossProbability":0.0,"delayRounds":0,"packetExpirySeconds":86400,"seed":42}
```

PUT accepts any subset of these fields. Ranges: `defaultTtl` 1 to 32, `lossProbability` 0 to 1, `delayRounds` 0 to 20, `packetExpirySeconds` at least 1. Violations return 400.

11.11 GET /api/mesh/packets/{packetId}

Keys: `packetId`, `status`, `fingerprint`, `ciphertextPreview`, `ciphertextLength`, `createdAt`, `initialTtl`, `origin`, `finalOutcome`, `holders[]` (`deviceId`, `role`, `ttl`, `hops`, `receivedFrom`, `uploadState`, `uploadAttempts`, `expired`), `events[]` (`seq`, `atMillis`, `round`, `type`, `packetId`, `from`, `to`, `detail`) and `uploads[]` (`deviceId`, `outcome`, `reason`, `message`, `round`, `atMillis`). 404 for an unknown packet.

11.12 POST /api/bridge/ingest

Body is a `MeshPacket`: `{"packetId":"...","ttl":3,"createdAt":1791557257510,"ciphertext":"<Base64>"}` with `packetId` and `ciphertext` not blank, `ttl` at least 0 and `createdAt` present. Optional headers `X-Bridge-Node-Id` (default `unknown`) and `X-Hop-Count` (default 0).

Outcome	HTTP	Body
SETTLED	200	`{"outcome":"SETTLED","reason":null,"packetHash":"...","ledgerEntryId":1,"message":"Payment settled","retryable":false}`
DUPLICATE_DROPPED	200	`outcome` DUPLICATE_DROPPED, `ledgerEntryId` null
REJECTED	422	`outcome` REJECTED with `reason`, for example DECRYPTION_FAILED or STALE_TIMESTAMP
FAILED	503 + `Retry-After: 1`	`outcome` FAILED, `retryable` true
Malformed request	400	Standard error shape; nothing is claimed

11.13 Experiments

`GET /api/experiments/types` lists the six experiments with their parameter, default values and metrics. `GET /api/experiments` returns the last 20 results, newest first. `POST /api/experiments/run` takes `{"type":"PACKET_LOSS","runs":30,"seed":42,"values":[0,0.2,0.4]}` where `type` is CONNECTIVITY_DELAY, PACKET_LOSS, TTL, GOSSIP_ROUNDS, BRIDGE_AVAILABILITY or DUPLICATE_DELIVERY, `runs` is 1 to 100 and `values` (at most 12) are validated per experiment (for example loss in 0 to 1, TTL a whole number 1 to 32). The result contains `rows[]` (`param`, `samples`, `metrics`), the `metrics` catalogue and `parameters` (seed, runs per point, values and the fixed conditions) so the run can be reproduced. Invalid input returns 400.

12. Testing and verification

12.1 How to run the tests

```
./mvnw test
```

```
./mvnw test -Dtest=ConcurrentIngestionTest
```

A single test class can be selected with `-Dtest=ClassName`. The suite starts one Spring context (in-memory H2) shared by the integration classes and takes roughly 15 to 25 seconds.

12.2 What the tests cover

Test class	Count	Purpose
`HybridCryptoServiceTest`	8	Round trip; no plaintext in ciphertext; wire format length; fresh randomness; tamper in RSA part, IV, ciphertext and tag; invalid formats; wrong server key.
`IngestionTest`	21	Settlement; atomic debit/credit/ledger; invalid amounts, unknown accounts, same account, insufficient funds; stale, future and in-skew timestamps; the four duplicate cases; tamper; malformed packets; rollback and claim release after a mid-transaction failure; database-only duplicate detection; reset.
`ConcurrentIngestionTest`	5	3 bridges and 30 copies of one packet; six repackaged copies racing on one nonce; 90 concurrent deliveries over shared accounts with exact balances and conserved money; concurrent overdraft impossible.
`RetryableFailureTest`	3	Transient conflicts retried inside the backend; persistent conflict becomes FAILED with the claim released and a later retry succeeds; unexpected exception contained.
`MeshNetworkTest`	19	One hop per round; TTL decrement and exhaustion; reachability by TTL; duplicate suppression; retention while the bridge is offline; upload acknowledgement rules; loss keeps copies; delay; partition; expiry; determinism by seed.
`ApiTest`	20	Every endpoint and error case, including health, validation, 404/409, concurrent HTTP ingestion.
`ScenarioTest`	10	The eight demonstrations end to end over HTTP, checking real database rows.
`ExperimentTest`	10	Expected shape of each experiment, reproducibility, isolation from demo data, validation.
`DashboardTest`	2	`/` forwards to the dashboard; every element id used by `app.js` exists in `index.html` (guards dead controls).

Tests assert on repositories (balances, ledger rows, packet records, audit events), not only on HTTP responses. They map to the required matrix as follows: valid offline payment and gossip (`ScenarioTest`, `MeshNetworkTest`); bridge offline and restored (`MeshNetworkTest`, `ScenarioTest` scenario 1 and 6); settlement, replay and repackaged duplicate (`IngestionTest`); three concurrent submissions (`ConcurrentIngestionTest`, `ApiTest`); tamper, expired, malformed (`IngestionTest`, `HybridCryptoServiceTest`); TTL (`MeshNetworkTest`, scenario 5); insufficient funds, invalid account, invalid amount (`IngestionTest`); failed upload and database failure (`RetryableFailureTest`, `IngestionTest`); multiple concurrent payments (`ConcurrentIngestionTest`); reset (`IngestionTest`, `ApiTest`); API errors (`ApiTest`).

12.3 Verification results (actual executions)

Test category	Command or procedure	Actual result	Evidence or limitation
Full test suite	`./mvnw clean test`	**98 tests, 0 failures, 0 errors, 0 skipped. BUILD SUCCESS** (16 s on the final run)	Maven Surefire output; run on OpenJDK 26.0.1; class files are Java 17 (major version 61)
Package build	`./mvnw clean package -DskipTests`	BUILD SUCCESS; jar `target.nosync/upi-offline-mesh-1.0.0.jar`	Jar started with `java -jar`; `GET /` returned 200 text/html
Clean start	Copy of the project outside the workspace, then `./mvnw spring-boot:run`	Application started in about 4 s; `/api/health` returned UP	Manual, final code
Concurrency (idempotency)	`ConcurrentIngestionTest`, `ApiTest`	Exactly 1 SETTLED and N-1 DUPLICATE_DROPPED; ledger count and both balances verified	Database state asserted
Mutation check	Disabled the claim layer temporarily	17 tests failed (12 in `IngestionTest`, 5 in `ConcurrentIngestionTest`); original code restored	Shows the duplicate tests are not vacuous

Test category	Command or procedure	Actual result	Evidence or limitation
Frontend build / lint	Not applicable (static files)	`node --check app.js` passes	No bundler or linter is configured
Dashboard end to end (real mouse clicks)	Built-in browser: create payment, gossip x4, bridge online, upload	All 7 stages lit, "1 upload(s): 1 settled", balances 4,500 / 1,500, one ledger row	Manual; not part of the Maven suite
Guided scenarios in the browser	Hero button "Run all scenarios"	8 scenarios, 27 checks passed, 0 failed	Manual, final build
Responsive layout	Phone-width pane and a 375 px iframe	No horizontal overflow; sections reflow	Manual; desktop width verified earlier in the session
Experiment reproducibility	Same request (packet loss, seed 42, 30 runs) run twice, before and after a restart	Identical rows (100, 100, 100, 100, 60, 26.667 %)	Manual plus `ExperimentTest`
Persistence across restart	File-backed H2, settle, restart	Balances and ledger row preserved, no re-seeding	Manual
Not run	Automated browser tests (Playwright MCP server failed to connect); a real JDK 17 runtime; `mvnw.cmd` on Windows; Firefox/Safari; screen-reader testing	-	Limitations, see 15

12.4 Defects found and fixed during the final audit

Finding	Fix	Verified by
Shell commands with trailing `# comments` (in my instructions and the README) broke in interactive zsh.	Commands are now one per block without inline comments; README notes it.	Commands re-run exactly as written
The scenario "Run all" button still used an old CSS class and rendered unstyled.	Class renamed to the current `solid` button style.	Visual check; contract test for ids
The hero used `overflow: hidden`, which can still scroll programmatically: clicking "Start a payment" shifted the hero 48 px sideways.	Changed to `overflow: clip`.	`scrollLeft` measured 0 after clicking the anchor
Statistic strip had low contrast over the pale bottom of the sky gradient.	Darker translucent background.	Visual check on phone width
Browser kept a stale `app.js` after updates (heuristic caching).	`Cache-Control: no-cache` for static resources.	`curl -I` shows the header
No health endpoint for the "verify the backend is healthy" step.	Added `GET /api/health` with a test.	`ApiTest.healthReportsBackendDatabaseAndMeshStatus`
No guard against dead controls.	`DashboardTest` checks every id used by `app.js` exists in the page.	Test passes
Experiments panel stayed empty after a reload even when earlier results existed.	The latest result is shown on load.	Visual check
Build output corrupted by iCloud sync (`* 2.class`).	Maven output directory is `target.nosync`.	Clean builds repeatedly successful

13. Troubleshooting

Problem	Likely cause	Solution
Startup fails: port in use	Another process on 8080	`PORT=9090 ./mvnw spring-boot:run` or stop the other process (`lsof -ti :8080`).
Dashboard blank or unstyled	Stale browser cache or the application is not running	Reload; check `curl -s localhost:8080/api/health` ; static files are served with `no-cache` .
"Backend unreachable" banner	Application stopped or crashed	Start it again; the page recovers by itself within 2 seconds.
Controls greyed out	The control is invalid right now	Hover for the tooltip and read the hint under the controls: e.g. no packet yet, no bridge online, sender equals receiver.
Upload shows DUPLICATE	Another bridge already settled the same bytes	Expected behaviour: only the first delivery settles.
Upload shows REJECTED / DECRYPTION_FAILED	Packet tampered, or created before a restart (new RSA key)	Create a fresh payment in Normal mode.
REJECTED / NONCE_REUSED	The same payment was re-encrypted ("repackaged")	Expected: a logical payment settles once.
REJECTED / STALE_TIMESTAMP	Expired mode, or the freshness window was lowered	Use Normal mode, or check `upi.mesh.freshness-window-seconds` .
REJECTED / INSUFFICIENT_FUNDS	Sender balance below amount at settlement time	Reduce the amount or settle another payment first.
Unexpected balances	Earlier scenarios or experiments changed state	Press "Reset to baseline" (two clicks). Experiment traffic never touches demo balances.
Stale UI numbers	A request failed silently in another tab	Reload the page; all values come from the backend.
400 on a request you built by hand	Validation failed	Read `details` in the JSON error, e.g. `"amount: must be greater than 0"` .
`wrong name: ...Test 2` during tests	iCloud conflict copies	`./mvnw clean test` .
Database initialisation error (file mode)	Old or corrupt `./data` files	Stop the app, delete the `data` folder, start again.

14. Viva preparation

Why is a mesh useful when direct connectivity is unavailable?

A mesh lets the packet reach a gateway through other devices that happen to have connectivity, instead of requiring the sender itself to be online. In the demo, Alice has no internet but her packet travels relay-1, relay-2 to bridge-1 (`Topology.demo()`), and bridge-1 uploads it.

Why is store and forward needed?

Links are intermittent: a bridge may be offline when the packet arrives. Each device stores the packet (`StoredPacket`) until a later opportunity. A failed forward keeps the sender's copy, and an offline bridge keeps the packet queued until `online` becomes true. Without storage the payment would be lost whenever the next hop was unavailable.

How does gossip forwarding work in this project?

In each round every active device offers every packet it holds (TTL above 0, not expired) to each active neighbour that does not have it. The receiver stores a copy with TTL-1. A start-of-round snapshot makes the spread exactly one hop per round (`MeshNetwork.gossipRound`). It is neighbour-based, not a global broadcast; the `full-mesh` topology exists only for comparison.

Why is TTL required?

Replication floods the network. TTL bounds how far a packet can travel so it cannot circulate forever. The cost is reachability: with TTL 2 the packet stops at relay-2 and relay-3 and never reaches a bridge three hops away (status TTL_EXHAUSTED, scenario 5, experiment "TTL").

Why hybrid encryption?

RSA can only encrypt small data and is slow; AES handles any size but needs a shared key. We encrypt the JSON with a fresh AES-256-GCM key and wrap only that key with RSA-OAEP for the server (`HybridCryptoService`). Only the backend holds the private key, so relays cannot read the payment.

Why does AES-GCM detect tampering?

GCM is authenticated encryption: it produces a 128-bit tag over the ciphertext. Changing any bit makes tag verification fail with `AEADBadTagException`, so no plaintext is released. `HybridCryptoServiceTest.flippingAnyRegionOfTheCiphertextIsRejected` flips a byte in the RSA part, IV, ciphertext and tag and each is rejected.

Why does RSA protect the AES key rather than the payload?

The AES key is 32 bytes and fits easily in one RSA-OAEP block of a 2048-bit key (256 bytes), whereas the payload would need chunking and would be slow. Each packet has its own random key, so exposing one key exposes only one packet.

Why does encryption not prevent replay attacks?

A replayed packet is still a perfectly valid ciphertext; decrypting it succeeds. Preventing replay needs state or freshness: the ciphertext-hash claim for identical bytes, a unique nonce for logically identical payments, and the `signedAt` freshness window (`BridgeIngestionService`, `LedgerEntry`).

Why is ciphertext hashing alone insufficient for logical duplicate protection?

Re-encrypting the same instruction uses a new AES key and IV, producing a different ciphertext and hash. The hash check would accept it. The nonce is inside the authenticated payload and is the same in both packets, so the unique constraint `uk_ledger_nonce` rejects the second settlement (`IngestionTest.case2_...`).

How does idempotency work?

The backend computes SHA-256 of the ciphertext and atomically claims it: an in-memory `putIfAbsent` as a fast path, then an INSERT into `packet_records` with a UNIQUE `packet_hash` in its own committed transaction. The first claimant processes; every other copy gets DUPLICATE_DROPPED. A transient failure releases the claim so the packet can be retried (`PacketClaimService`).

How is concurrent duplicate settlement prevented?

Three layers: the claim (only one thread proceeds), the unique nonce and packet hash in `ledger_entries`, and row locks (`SELECT ... FOR UPDATE`) on both accounts inside one transaction. `ConcurrentIngestionTest` fires 3 and 30 simultaneous copies and checks one ledger row and exact balances.

What happens when a bridge is offline?

The packet reaches the bridge through gossip and stays there with upload state PENDING (status AT_BRIDGE). `pendingUploads()` skips bridges that are not online and in the mesh, so nothing is submitted and nothing settles.

How does the system resume after connectivity returns?

When the bridge goes online the next upload submits the same bytes. If the answer is SETTLED, DUPLICATE_DROPPED or REJECTED the copy is acknowledged; if it is FAILED the copy stays queued for the next upload. The sender does nothing (scenario 6).

How are account and ledger consistency maintained?

`SettlementService.settle` is one `@Transactional` method: it locks both accounts in alphabetical order, checks the balance, debits, credits, inserts the ledger entry with `saveAndFlush` and marks the packet SETTLED. Any failure, including a unique-constraint violation, rolls back everything (`IngestionTest.failureAfterDebitRollsBackBalancesAndReleasesTheClaim`). Money uses `BigDecimal`.

What is simulated and what is implemented directly?

Implemented directly: all cryptography, hashing, idempotency, locking, transactions, the REST API and the database. Simulated: the radio layer, device mobility, the phones themselves, the bridge-to-backend hop (in-process call into the same service) and the bank (fictional accounts).

Why is this not a real UPI system?

There is no NPCI or bank integration, no real identity, authentication or PIN verification, no regulated settlement and no real money. Key management and transport security are demo-grade. The project demonstrates the networking and consistency principles an offline extension would need.

What does "exactly once" mean here?

One successful ledger effect per logical payment despite repeated delivery. It does not mean the network transmits a packet exactly once; the mesh deliberately delivers duplicates.

What is the difference between REJECTED and FAILED?

REJECTED is permanent for those bytes (bad crypto, stale, business rule) and the bridge must not retry. FAILED is transient (lock conflict, unexpected error): the claim is released so a retry can succeed. This prevents a packet from being stuck in an ambiguous state.

How do the experiments stay reproducible?

Each run builds a private `MeshNetwork` with a seeded `java.util.Random`; the experiment result records seed, runs per value, values and fixed conditions. The same request returns identical rows (checked by `ExperimentTest` and by re-running after a restart).

Where is the limitation of offline payments the project cannot solve?

Double spending: an offline sender cannot prove it has funds. The backend settles in arrival order and rejects later payments that the balance cannot cover. Solving it needs a pre-funded offline wallet or secure hardware.

15. Known limitations and future work

15.1 Limitations that remain after the final audit

Limitation	Impact	Priority of future work
No real wireless layer (Bluetooth/Wi-Fi Direct), discovery, mobility or power model	Results show protocol behaviour, not radio performance.	High for a real deployment
No authentication of users, bridges or requests; no rate limiting; no TLS; open demo controls and H2 console	Must not be exposed to an untrusted network.	High
RSA key regenerated at every start and held in memory	Packets cannot survive a restart; no rotation or forward secrecy.	High (HSM/KMS)
Offline double spending cannot be prevented	Later payments from an exhausted balance are rejected at settlement.	High (pre-funded wallet)
Bridge upload is an in-process call in the simulator	The REST endpoint is tested but not used by the simulator over a network.	Medium
Mesh state and experiment history are in memory	Lost on restart; mesh is not persisted.	Medium
Idempotency fast path is a JVM map	A multi-instance deployment relies on the database constraint (which works) plus a shared cache such as Redis.	Medium
Nonces are retained as long as the ledger table lives	Production should retire nonces after the freshness window plus a margin.	Low
Loss model is i.i.d.; gossip order deterministic; unlimited storage	Simplifies analysis; real networks are burstier.	Low
No PIN, identity or regulatory integration	Out of scope for a networking prototype.	Out of scope
Browser end-to-end tests are manual, not in the Maven suite	UI regressions are caught only by `DashboardTest` and manual checks.	Medium

15.2 Verification limits

The final verification was run on OpenJDK 26.0.1 compiled with `--release 17`; it was not run on a JDK 17 runtime. `mvnw.cmd` on Windows, Firefox, Safari and screen-reader behaviour were not tested. The Playwright MCP server failed to connect in the session, so browser checks were done with the built-in browser tool instead of an automated suite.